🟢 **TASK 1: Student Information System (Basics)**

📌 **Problem**

Design a class Student that stores:

- Name
- Roll number

Provide:

- A constructor to initialize values
- A method to display student details

👉 Create at least 2 student objects and display their details.

🟢 **TASK 2: Rectangle Calculator (Constructor + Methods)**

📌 **Problem**

Create a class Rectangle:

- Attributes: length, width
- Methods:
    - calculate area
    - calculate perimeter

👉 Demonstrate using user input.

🟡 **TASK 3: Bank Account System (Encapsulation)**

📌 **Problem**

Create a class BankAccount:

- Private attribute: balance
- Methods:
    - deposit(amount)

    - withdraw(amount)
    - show_balance()

👉 Conditions:

- Prevent withdrawal beyond balance
- Prevent negative deposits

👉 Use a private helper method to display balance.

🟡 **TASK 4: Employee Salary System (Inheritance)**

📌 **Problem**

Create a base class Employee:

- Attributes: name, base salary
- Method: calculate_salary()

Create child classes:

- Developer → add coding bonus
- Manager → add management bonus

👉 Override calculate_salary() in each child class.

🟡 **TASK 5: Multilevel Banking System**

📌 **Problem**

Design a multilevel inheritance system:

- Class 1: Account → holds balance
- Class 2: DepositAccount → handles deposit
- Class 3: TransactionAccount → handles withdraw & display

👉 Ensure:

- Balance is private

- Proper method flow across levels

🟠 **TASK 6: Shape System (Abstraction)**

📌 **Problem**

Create an abstract class Shape:

- Abstract method: area()

Create:

- Circle
- Rectangle

👉 Each class must implement area calculation.

🟠 **TASK 7: User Login System (Encapsulation + Abstraction)**

📌 **Problem**

Create class User:

- Private password
- Private method to verify password
- Public method:
    - login()

👉 Hide verification logic from user.

🟠 **TASK 8: File Processing System (Abstraction)**

📌 **Problem**

Create class FileProcessor:

- Public method: process_file()

Internally:

- Read file
- Analyze content

👉 Hide internal steps using private methods.

🔴 **TASK 9: Payment System (Polymorphism)**

📌 **Problem**

Create base class Payment:

- Method: pay(amount)

Create:

- UPI
- Card
- NetBanking

👉 Override pay() in each class.

🔴 **TASK 10: Operator Overloading System**

📌 **Problem**

Create class Point:

- Attributes: x, y

Overload:

- + operator → add two points

👉 Example:

(2,3) + (4,5) → (6,8)

🔴 **TASK 11: Product Comparison System**

📌 **Problem**

Create class Product:

- Attribute: price

Overload:

- > operator to compare products

🔴 **TASK 12: Smart Home System (Combined Concepts)**

📌 **Problem**

Design a smart home system:

- Base class: Device
- Derived classes:
  - Light
  - Fan
  - AC

👉 Requirements:

- Use inheritance
- Override method turn_on()
- Add abstraction for internal operations

🧠 **TASK 13: Library Management System (Mini Project)**

📌 **Problem**

Create a class Library:

- Private list of books

Methods:

- add book
- issue book
- return book

- display books

👉 Apply:

- Encapsulation
- Proper validation

🧠 **TASK 14: Log Analyzer Tool (Real-World)**

📌 **Problem**

Create class LogAnalyzer:

- Private log file

Methods:

- count errors
- display warnings

👉 Use:

- File handling
- Encapsulation

🧠 **TASK 15: ATM Simulation (Final Integrated Task)**

📌 **Problem**

Build an ATM system:

- Class: Account
- Features:
    - deposit
    - withdraw
    - check balance
    - PIN verification

👉 Apply:

- Encapsulation
- Abstraction
- Inheritance (optional)